



ShoppingCartSystem

OOP Project Report

Group members :

Huda Al-Halaf 44611642

Najd altubaiti 44610903

Introduction

This report provides a brief overview of the design choices behind the ShoppingCartSystem project, along with an explanation of how each required Object-Oriented Programming (OOP) concept was implemented. The project simulates a simplified online shopping flow where users can view products, add items to their cart, and generate an order. Detailed screenshots of the program's outputs are also included.

Design Overview

The system follows a straightforward domain structure:

- Store manages collections of Products and Customers.
- Each Customer owns a ShoppingCart and may create one or more Orders.
 - Product is designed as an abstract class, while PhysicalProduct extends it with additional fields such as brand and weight.
 - Borrowable is an interface implemented by Product to model availability behavior (checkout(), checkin(), isAvailable()).

Key design considerations:

- Separation of concerns across classes (store management, cart behavior, product modeling.)
- Strong relationships where lifecycles are dependent (composition), and flexible connections where objects can exist independently (aggregation).
- Simple, clear menu-based user interaction to demonstrate OOP functionality.

Implementation of Required OOP Concepts

Abstraction

The system uses the abstract class Product to represent shared properties of all items
Concrete implementation is provided in PhysicalProduct which adds physical attributes

Inheritance

PhysicalProduct extends Product

This demonstrates classical inheritance where the child class reuses and extends the behavior of the parent

Interface Implementation

The Borrowable interface defines required methods related to availability

Product implements Borrowable to provide concrete implementations of these methods

Polymorphism

The system stores objects as Product references while holding actual PhysicalProduct objects in lists such as

ArrayList in the Store catalog

ArrayList in the ShoppingCart

This allows dynamic dispatch of overridden methods like toString and checkOut

Collections Framework

The system uses ArrayList for the catalog and cart items

The system uses ArrayList for tracking customer orders

The system uses TreeSet for maintaining sorted customers

This demonstrates the use of OOP collections and custom comparison through compareTo in Customer

Custom Exceptions

The class ProductNotFoundException is thrown when the system fails to find a product by ID in the Store

This demonstrates exception handling and improves system robustness

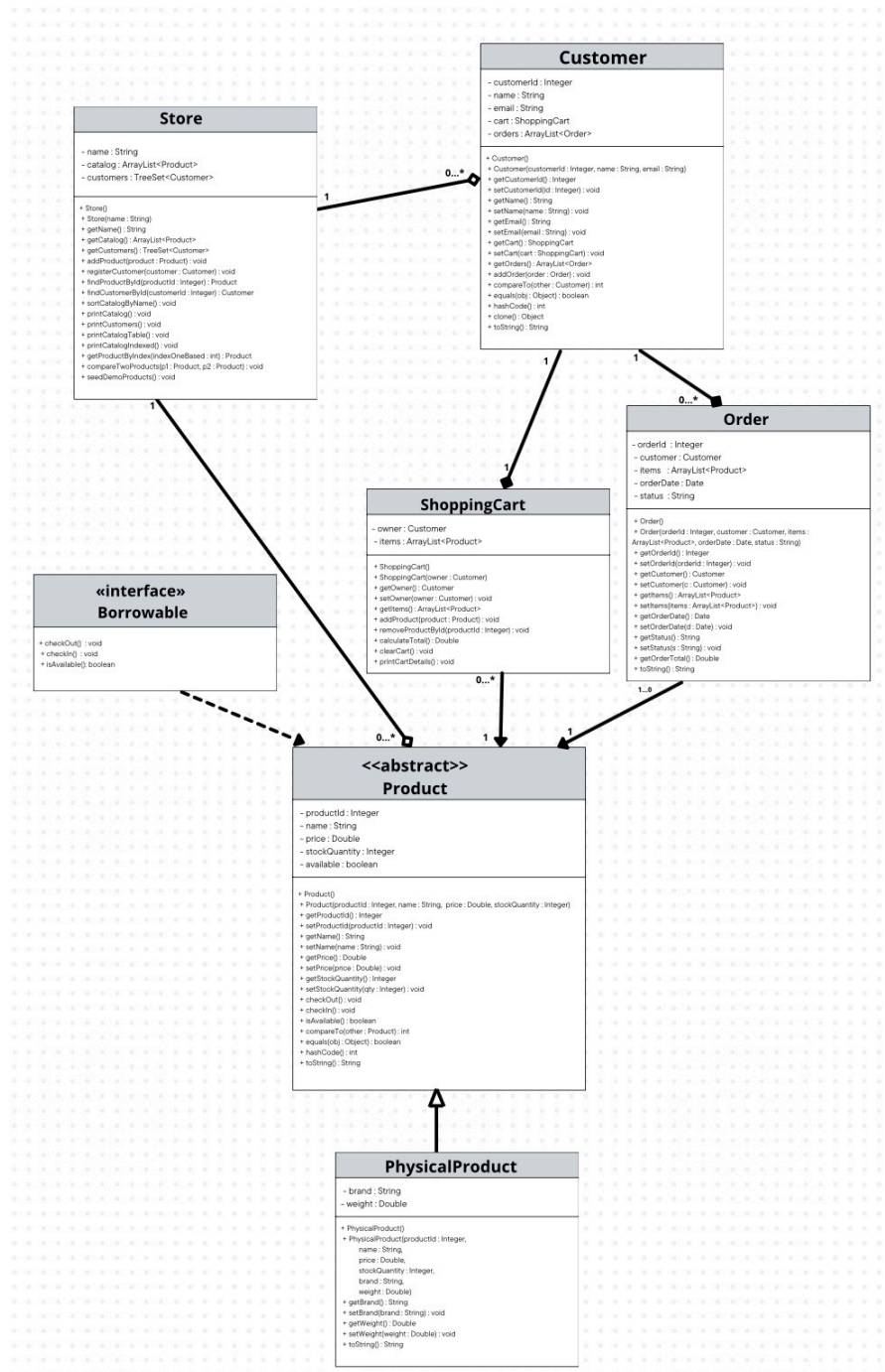
equals compareTo clone

These methods are used to support comparison of objects such as sorting customers and products

They are also used for checking equality like when preventing duplicate entries

They support object duplication when needed

UML Class Diagram



Inheritance: PhysicalProduct → Product

Interface Realization: Product → Borrowable

Aggregation: Store → Product, Store → Customer

Composition: Customer → ShoppingCart, Customer → Order

Associations: ShoppingCart ↔ Product, Order ↔ Product

Dependency: Store → ProductNotFoundException

Output Screenshots

Program Start

User enters ID, name, and email.

```
--- exec:3.1.0:exec (default-cli) @ ShoppingCartSystem ---  
=== Welcome to Shopping Cart System ===  
Enter your ID: 555  
Enter your name: huda  
Enter your email: huda@gmail.com  
? Welcome, huda!
```

Product List

A numbered catalog of available products.

```
=== Products ===  
1) Headphones | ID=4 | price=180.0 | stock=30 | brand=Sony | weight=0.3  
2) Keyboard | ID=5 | price=120.0 | stock=25 | brand=Keychron | weight=0.7  
3) Laptop | ID=1 | price=3500.0 | stock=10 | brand=Lenovo | weight=2.5  
4) Smartphone | ID=2 | price=2500.0 | stock=15 | brand=Samsung | weight=0.5  
5) Wireless Mouse | ID=3 | price=80.0 | stock=50 | brand=Logitech | weight=0.2  
-  
Enter product number to add (0 to stop):
```

Output Screenshots

Adding to Cart

After selecting products by number.

```
--- exec:3.1.0:exec (default-cli) @ ShoppingCartSystem ---
=== Welcome to Shopping Cart System ===
Enter your ID: 555
Enter your name: huda
Enter your email: huda@gmail.com
? Welcome, huda!

=== Products ===
1) Headphones | ID=4 | price=180.0 | stock=30 | brand=Sony | weight=0.3
2) Keyboard | ID=5 | price=120.0 | stock=25 | brand=Keychron | weight=0.7
3) Laptop | ID=1 | price=3500.0 | stock=10 | brand=Lenovo | weight=2.5
4) Smartphone | ID=2 | price=2500.0 | stock=15 | brand=Samsung | weight=0.5
5) Wireless Mouse | ID=3 | price=80.0 | stock=50 | brand=Logitech | weight=0.2

Enter product number to add (0 to stop): 1
Added product to cart: Headphones

Enter product number to add (0 to stop): 2
Added product to cart: Keyboard

Enter product number to add (0 to stop): 5
Added product to cart: Wireless Mouse

Enter product number to add (0 to stop): 0

--- Actions ---
```

Viewing Cart

Items and total price displayed.

```
--- Actions ---
1) Add more products
2) View cart
3) Checkout
4) Exit
Choose: 2
Shopping Cart for: huda
PhysicalProduct{productId=4, name='Headphones', price=180.0, stockQuantity=30, available=true, brand='Sony', weight=0.3}
PhysicalProduct{productId=5, name='Keyboard', price=120.0, stockQuantity=25, available=true, brand='Keychron', weight=0.7}
PhysicalProduct{productId=3, name='Wireless Mouse', price=80.0, stockQuantity=50, available=true, brand='Logitech', weight=0.2}
Total: 380.0

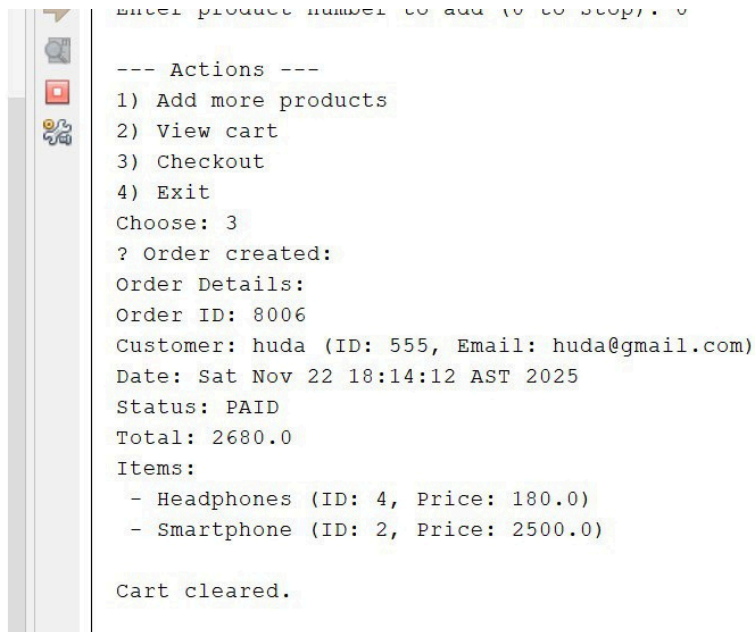
--- Actions ---
1) Add more products
2) View cart
3) Checkout
4) Exit
Choose: 4
Goodbye!

-----
BUILD SUCCESS
```

Output Screenshots

Checkout

Order created and cart cleared.



```
Enter product number to add (0 to stop): 0  
--- Actions ---  
1) Add more products  
2) View cart  
3) Checkout  
4) Exit  
Choose: 3  
? Order created:  
Order Details:  
Order ID: 8006  
Customer: huda (ID: 555, Email: huda@gmail.com)  
Date: Sat Nov 22 18:14:12 AST 2025  
Status: PAID  
Total: 2680.0  
Items:  
- Headphones (ID: 4, Price: 180.0)  
- Smartphone (ID: 2, Price: 2500.0)  
  
Cart cleared.
```

The user selected Checkout from the actions menu.

The system created a new Order using the items inside the ShoppingCart.

The order was assigned a random order ID, linked to the correct customer, given the current date and time, and marked as PAID.

After the order was created, the system cleared the cart to indicate that the purchase was completed.

Conclusion

The project successfully demonstrates the key principles of Object-Oriented Programming through a clean, modular design. It integrates abstraction, inheritance, interfaces, collections, custom exceptions, and polymorphism to create a functional and extendable shopping system. The final implementation is simple, readable, and aligns with the course's OOP requirements.